

CS 431: Computer and Network Security Assignment 2

rsa-state-of-affairs — python3 solution.py 1

Flag : CNS431{r5a_5t4t3_0f_4ff41r5}

We notice that the public exponent is very small, ($e = 5$) and the modulus, n is very large. As told in class, small public exponents are vulnerable, as the modulus becomes redundant, and the ciphertext c is just n^5 . We observed that the message m was small enough such that $m^e < n$. To decrypt, we computed the integer 5th root of c using [gmpy2.iroot](#). The resulting integer, when converted to bytes, revealed the plaintext password, which we sent to the server to obtain the flag.

hash-extension — python3 solution.py 2

Flag : 431fl@g{D3@D3Xt3ns!0nL3ngth}

Here we can apply a [Length Extension Attack](#) on SHA-256 (because of Cipher Block Chaining). The server signs cookies using $H(secret||message)$. As shown in class, we can compute $H(secret||message||padding||extension)$ given only $H(secret||message)$ and the length of the secret + message, without knowing the secret itself. We used the [hlexend](#) library to append the string `admins`, to the valid cookie `user=cns431...`. We forged the new signature and the corresponding extended message (including the necessary padding), which tricked the server into believing we are admins.

Login Error — python3 solution.py 3

Flag : cnsctf{23110146.1770205562.412272_2ef1b9e9ce9e0ab03cb536b65ebed89d0e21aad07d28e51d2346ce0827a7edd9}

Here we can perform the [Bit Flipping Attack](#) in AES-CBC mode. In this mode, decryption of a block involves XORing with the previous ciphertext block (or IV for the first block). Changing a bit in the IV (or C_{i-1}) flips the corresponding bit in the plaintext P_i . The server provided encrypted credentials `user:c?s` and `pass:c?f`. We needed to change `?` to `n` and `t` respectively (found by bruteforcing). We exploited this by XORing the specific bytes in the IV (since the message was in the first block) with $mask = ('?' \oplus 'n')$ and $mask = ('?' \oplus 't')$. Sending this modified ciphertext to the server allowed us to log in successfully and retrieve the flag.

Note: An observation, the flag had the format which will match this regex: `cnsctf{\d{8}\_\d{10}\.\d+\_([0-9]|[a-f]){64}\}`. Also, the first part is the rollnumber, the middle part is the timestamp, and the last part looks like the hash (probably SHA-256).

Alice's Signature Mix-Up — python3 solution.py 4

Flag : CNS431{@11c3_m@d3_@_k3y_mist@k3}

We tried many different types of cryptographic hashes but none worked. We then took the hint from the CTF server, and found it was a very simple additive hash. Hence, we computed this hash of the message by summing up all byte values and taking modulo $2^{32} - 1$ (in code it can be done by taking bitwise and with $2^{32} - 1$). Next we signed this hash using Alice's public key (e, n) instead of her private key. The server accepted this as a valid signature, and gave us the flag.

```
ts/Assignment 2$ python3 solution.py 1
Problem: rsa-state-of-affairs
True
6093028074164187238163437059075857402400554033884189534499042015660476877336
9280381274743986965646834968469200078865700052232117387897640855805898020227
17965574441371602701042167453573974650735362939498006613865012
[+] Opening connection to 10.0.118.104 on port 1234: Done
[+] Receiving all data: Done (2.57KB)
[*] Closed connection to 10.0.118.104 port 1234
Flag: CNS431{r5a_5t4t3_0f_4ff41r5}

ts/Assignment 2$ python3 solution.py 2
Problem: hash-extension
[+] Opening connection to 10.0.118.104 on port 4310: Done
[+] Receiving all data: Done (195B)
[*] Closed connection to 10.0.118.104 port 4310
Flag: 431fl@g{D3@D3Xt3ns!0nL3ngth}
```

```
ts/Assignment 2$ python3 solution.py 3
Problem: Login Error
Trying user:cns | pass:ctf
Flag: cnsctf{23110146.1770205562.412272_2ef1b9e9ce9e0ab03cb536b65ebed89d0e21
aad07d28e51d2346ce0827a7edd9}

ts/Assignment 2$ python3 solution.py 4
Alice's Signature Mix-Up
[+] Opening connection to 10.0.118.104 on port 4444: Done
[+] Receiving all data: Done (92B)
[*] Closed connection to 10.0.118.104 port 4444
Flag: CNS431{@11c3_m@d3_@_k3y_mist@k3}
```

Source Code: solution.py

```
import sys
from gmpy2 import iroot
import binascii
import pwn # https://github.com/Gallopsled/pwntools
import re

# pip install gmpy2 pwntools

def problem1():
    """Reference:
    https://github.com/gmpy2/gmpy2
    """
    print("Problem: rsa-state-of-affairs")
    c = int("""83978071260346981418948702731606989705805594626377490
63850451431139297578400476522913163177634017340306337512
27130989020645561695820984721967201500422490707846159216
57461169469547342737012321094750788978007048664534393372
94167762610969388300160263884345203773946188532683434548
64689930499235696244712849070664244259340919798747141271
92613155295325360229531350177615405468722476378970820108
99471009636885853273822361118949789438199529052819145978
25113458757003211719243346387803990048584591683635917436
63865288404249051843788207366447332782737149165684514419
09311272117773515784513941463207969016416165188903980077
06274841678968433629636172561094053902837904992556228639
21756542015092652729015922797956498475990216040589415389
35295849615748603791882890287576074582777748112220757615
25919817983156848841544788481265439025190175634394567937
63840044686592812062953260025095363056378198265119769075
47251864001229438341563687335803416115326142703945949523
09243999689382024854263811150807886205874483189490040723
77175759014365670338040945847852269935194658083938022535
23448832""").replace("\n", "").replace(" ", ""))
    m, exact = iroot(c, 5)
    print(exact)
    print(m)

    msg = int(m).to_bytes((m.bit_length() + 7) // 8, "big")
    # print(msg)
    r = pwn.remote("10.0.118.104", 1234)
    # print(r.recv().decode())
    r.sendline(msg.decode("utf-8").split(": ")[-1].encode())
    response = r.recvall()
    resp = response.decode()
    # print(resp)
    print(f"Flag: {resp.splitlines()[-1].split(':')[1]}")

def problem2():
    """References:
    https://crypto.stackexchange.com/questions/3978/understanding-the-length-extension-attack
    https://github.com/stephenbradshaw/hlextend
    """
    import problems.lib.hlextend as hlextend

    print("Problem: hash-extension")
    r = pwn.remote("10.0.118.104", 4310)
    r.recv()
    msg: str = r.recv().decode()
```

```

secret_key_len = 16
t = (
    msg.split("Here is your cookie as plaintext:")[1]
    .strip()
    .split("And here it is as hex")
)
cookies = t[0].strip()
rest = t[1].split("\n\n")
# hex_cookie = rest[0].split(':')[1].strip()
sign_cookie = (
    rest[1].split("The signature of your cookie is:")[1].split("\n\n")[0].strip()
)
append_data = b"admins,"
sha = hlexend.sha256()
forged_msg = sha.extend(append_data, cookies.encode(), secret_key_len, sign_cookie)
forged_sig = sha.hexdigest()
forged_cookie = binascii.hexlify(forged_msg)
r.sendline(forged_cookie)
_ = r.recv().decode()
r.sendline(forged_sig.encode())
last = r.recvall().decode()
print(f"Flag: {last.splitlines()[-2]}")

def problem3():
    """References:
    https://pentesterlab.com/glossary/cbc-bit-flipping
    """
    pwn.context.log_level = "critical"
    print("Problem: Login Error")
    alphabets = "abcdefghijklmnopqrstuvwxyz"
    done = False
    for c1 in alphabets:
        if done:
            break
        for c2 in alphabets:
            if done:
                break
            print(f'Trying user:c{c1}s | pass:c{c2}f\r',end='')
            r = pwn.remote("10.0.118.104", 9999)
            r.recv()
            msg: str = r.recv().decode()
            og_user, og_pass = map(str.strip, msg.split("\n\n")[1].splitlines())

            def xor_string(hex_str, index, old_char, new_char):
                data = bytearray.fromhex(hex_str)
                xor_diff = ord(old_char) ^ ord(new_char)
                data[index] = data[index] ^ xor_diff
                return data.hex()

            mod_user = xor_string(og_user, 6, "?", c1)
            mod_pass = xor_string(og_pass, 6, "?", c2)
            r.sendline(b"23110146")
            r.recv()
            r.sendline(mod_user.encode())
            r.recv()
            r.sendline(mod_pass.encode())
            try:
                print(f"\nFlag: {r.recvall().decode().strip().split(': ')[1]}")
                done = True
            except:
                pass

```

```

def problem4():
    """References:
    Used a hint for figuring out the hash function.
    """
    print("Alice's Signature Mix-Up")
    r = pwn.remote("10.0.118.104", 4444)
    r.recv()
    msg: str = r.recv().decode()
    mt = re.findall(r'n = (\d+)\ne = (\d+)\nSecret Message:\n(.*)\n', msg)
    n = int(mt[0][0])
    e = int(mt[0][1])
    secret = str(mt[0][2]).encode()
    h = 0
    for b in secret:
        h = (h + b) & (2**32 - 1)
    data = int.from_bytes(h.to_bytes(4, "big"), byteorder="big")
    sig_int = pow(data, e, n)
    # print(str(sig_int).encode())
    r.sendline(str(sig_int).encode())
    m = r.recvall().decode()
    print(f"Flag: {m.split(': ')[-1]}")

if __name__ == "__main__":
    n = sys.argv
    if len(n) != 2:
        print(f"Usage solution.py <problem_id>")
        exit()
    prob = int(n[1])
    exec(f"problem{prob}()")

```